

Temperature Measurements using DS18B20 Sensor



Introduction:

Accurate temperature measurements are essential in various applications. In this tutorial, we will explore how to interface a DS18B20 digital temperature sensor with an Arduino board. We'll delve into the sensor's advantages, shortcomings, and methods to enhance its accuracy through calibration.

Objectives:

- Understanding DS18B20 Sensor:** Learn about the DS18B20 sensor, its advantages, and limitations.
- Hardware Integration:** Connect the DS18B20 sensor to an Arduino board.
- Calibration Techniques:** Calibrate the sensor to improve temperature measurement accuracy.
- Enhancing Accuracy:** Explore methods to enhance sensor accuracy through techniques like averaging and proper placement.

DS18B20 Sensor Overview:

Advantages:

- **High Accuracy:** DS18B20 offers precise temperature readings.
- **Simplicity:** Simple one-wire communication and digital output.
- **Wide Operating Range:** Suitable for various environments.

Shortcomings:

- **Inherent Inaccuracies:** Calibration often necessary for accurate readings.

Hardware Components:

- Arduino board (e.g., Arduino Uno)
- DS18B20 digital temperature sensor
- 4.7kΩ resistor (pull-up resistor) **ONLY if it didn't come connected already**
- Jumper wires
- Breadboard

Circuit Connections:

- DS18B20 Sensor to Arduino:

- VCC to 5V on Arduino
- GND to GND on Arduino
- DQ (Data) to digital pin 2 on Arduino with a 4.7kΩ pull-up resistor connected between DQ and 5V.

Installing Libraries:

- Open Arduino IDE.
- Go to Sketch -> Include Library -> Manage Libraries...
- Search for "OneWire" and "DallasTemperature" libraries and install them.

Arduino Code:

```
#include <OneWire.h>
#include <DallasTemperature.h>

#define ONE_WIRE_BUS 2 // Digital pin #2
OneWire oneWire(ONE_WIRE_BUS);
DallasTemperature sensors(&oneWire);

float offsetCorrection = 4.5; // Offset correction value in degrees Celsius

void setup() {
  Serial.begin(9600);
  sensors.begin();
  sensors.setResolution(12); // Set the resolution to 12 bits
}

void loop() {
  sensors.requestTemperatures(); // Send command to get temperatures
  delay(750); // Wait for the conversion to finish (for 12-bit resolution)

  float temperatureC = sensors.getTempCByIndex(0) + offsetCorrection; // Apply offset correction
  Serial.print("Temperature is: ");
  Serial.println(temperatureC, 2); // Print temperature with 2 decimal places
  delay(1000); // Wait for a second before the next reading
}
```

Calibration Process:

1. Upload the Code:

- Connect Arduino to your computer.
- Upload the provided Arduino code to the board.

2. Open Serial Monitor:

- Go to Tools -> Serial Monitor or press Ctrl + Shift + M.
- Note down the temperature readings displayed.

3. Calibration:

- Use a reliable thermometer to measure the actual temperature.
- Compare it with sensor readings in the Serial Monitor.
- Adjust `offsetCorrection` value in the code to match sensor readings with the actual temperature (e.g., if sensor reads 22.5°C but actual is 25°C, set offsetCorrection to 2.5).

4. Repeat Calibration (if necessary):

- If readings are inaccurate, repeat the process until sensor provides precise temperature measurements.

Enhancing Accuracy:

1. Temperature Averaging:

- Take multiple readings and average them to minimize minor fluctuations.

2. Proper Sensor Placement:

- Avoid heat sources or direct sunlight to prevent skewed readings.

3. Electromagnetic Interference Shielding:

- Shield the sensor to prevent erroneous readings due to electromagnetic interference.

Conclusion:

By following this tutorial, you've learned how to interface a DS18B20 digital temperature sensor with an Arduino board, calibrate the sensor, and enhance its accuracy. Understanding the sensor's intricacies and implementing calibration techniques empower you to obtain precise temperature data for your specific applications.

Happy experimenting!